



Universidad Veracruzana

Facultad de Estadística e Informática

Lic. en Ing. de Ciberseguridad e Infraestructura de Cómputo

Detección de tráfico anómalo en dispositivos IoT mediante aprendizaje automático

Experiencia Educativa:
Aprendizaje Máquina

Docente:
Mtro. Millan Martinez Max William

Presenta (n):
Jorge Alberto Hernández Hernández
Katya Cecilia Cruz Zamora
Israel Zajith Martínez Alarcón
Luis Daniel Cortés Hernández
Yael Fernando Saldaña Lopez

08 de junio de 2026

Índice

1. Introducción.....	3
2. Evaluación comparativa de los modelos.....	3
2.1. Métricas de rendimiento.....	5
2.2. Matrices de confusión e interpretación de errores.....	7
2.2.1. Matriz de confusión de K-Means multiclase.....	7
2.2.2. Matriz de confusión de K-Means binario.....	9
2.2.3. Matriz de confusión de Gaussian Naive Bayes.....	10
2.2.4. Matriz de confusión de Isolation Forest multiclase adaptado.....	11
2.2.5. Comparación general de las matrices.....	12
2.3. Curvas ROC y análisis AUC.....	12
3. Selección del modelo y justificación operativa.....	14
3.1. Modelo seleccionado para implementación.....	14
3.2. Trade-off operativo: Modelo Escudo vs Modelo Balanceado.....	14
4. Limitaciones y mejoras futuras.....	15
4.1. Limitaciones técnicas del análisis.....	15
4.2. Propuesta de mejoras y trabajo futuro.....	16
5. Conclusiones.....	18
6. Repositorio del proyecto.....	19
7. Referencias.....	19

1. Introducción

En la actualidad, los dispositivos de Internet de las Cosas, conocidos como IoT, forman parte de la vida cotidiana de millones de personas. Cámaras de seguridad, asistentes de voz, focos inteligentes, aspiradoras automatizadas, cerraduras electrónicas, sensores domésticos, etc pson ejemplos de tecnologías diseñadas para facilitar actividades diarias, mejorar la comodidad del usuario y permitir la automatización de espacios personales, académicos, industriales y empresariales. Sin embargo, esta conectividad permanente también introduce nuevos riesgos de ciberseguridad, ya que muchos de estos dispositivos cuentan con recursos limitados, configuraciones inseguras, credenciales débiles o mecanismos de protección insuficientes.

El problema se vuelve especialmente delicado cuando un dispositivo IoT es comprometido por un atacante. A diferencia de una computadora tradicional, muchos dispositivos inteligentes operan de forma continua y silenciosa, por lo que una intrusión puede pasar desapercibida durante largos periodos. Un dispositivo atacado puede ser incorporado a una botnet, es decir, una red de dispositivos comprometidos controlados de manera remota para ejecutar actividades maliciosas. Entre estas actividades se encuentran ataques de denegación de servicio distribuido, escaneo de redes, envío masivo de tráfico y participación en campañas automatizadas de intrusión. Botnets como Mirai y BASHLITE han demostrado que los dispositivos IoT pueden convertirse en una amenaza significativa para la disponibilidad, privacidad y seguridad de las redes.

Desde el punto de vista de la privacidad, el compromiso de dispositivos IoT resulta crítico. Una cámara, un asistente de voz o un sensor doméstico vulnerado puede exponer información personal, hábitos de comportamiento, ubicación, rutinas familiares o datos sensibles de los usuarios. Por ello, la detección temprana de tráfico anómalo en dispositivos IoT representa una tarea importante dentro de la ciberseguridad moderna. No basta con proteger equipos tradicionales; también es necesario analizar el comportamiento de dispositivos conectados y detectar señales que indiquen actividad maliciosa.

Para abordar este problema, el aprendizaje automático representa una alternativa útil, ya que permite analizar grandes volúmenes de datos y reconocer patrones que podrían no ser evidentes mediante una revisión manual. En el contexto de redes IoT, los modelos de aprendizaje automático pueden trabajar con características estadísticas del tráfico, como volumen, promedio, variación, jitter y dispersión, para identificar comportamientos que se alejan de la actividad normal de un dispositivo. Esto resulta relevante porque un ataque no siempre se manifiesta de forma evidente para el usuario, pero sí puede modificar el comportamiento del tráfico de red.

En este proyecto se trabaja con el dataset N-BaIoT, un conjunto de datos orientado al análisis de ataques de botnets en dispositivos IoT. A partir de sus características estadísticas, se busca evaluar si es posible distinguir entre tráfico benigno y tráfico asociado a botnets mediante modelos de aprendizaje automático. El objetivo principal es analizar la utilidad de estos modelos como apoyo para la detección temprana de tráfico anómalo en entornos IoT, considerando que una detección oportuna puede reducir el impacto de ataques automatizados sobre la privacidad, disponibilidad y seguridad de las redes.

2. Evaluación comparativa de los modelos

La evaluación de los modelos se realizó con el objetivo de analizar su capacidad para distinguir entre tráfico benigno y tráfico asociado a ataques de botnets en dispositivos IoT. Para ello, se utilizaron características estadísticas extraídas del tráfico de red del dataset N-BaIoT, el cual contiene tráfico real de dispositivos IoT infectados con botnets como Mirai y BASHLITE. De acuerdo con la descripción del repositorio UCI, este conjunto de datos fue construido a partir de tráfico de nueve dispositivos IoT comerciales y está asociado a tareas de clasificación y clustering.

En este proyecto se analizó específicamente el dispositivo **Provision PT-737E**, una cámara de vigilancia incluida dentro del dataset. Se seleccionó este tipo de dispositivo porque las cámaras IP son comunes en hogares, oficinas e instituciones, y su compromiso puede afectar directamente la privacidad de los usuarios y la seguridad de la red.

El dataset original cuenta con 7,062,606 registros y 115 variables atributo. Sin embargo, para este análisis se trabajó con un subconjunto de 80,000 registros distribuidos en cinco clases: una clase de tráfico benigno y cuatro clases de ataques de botnets seleccionadas de las diez que contiene el dataset original.

Clase	Registros utilizados
benign	40,000
gafgyt.scan	10,000
gafgyt.udp	10,000
mirai.scan	10,000
mirai.udp	10,000
Total	80,000

Aunque el conjunto completo contiene 115 variables, se seleccionaron cinco características estadísticas del tráfico de red. Estas variables fueron utilizadas como entrada para los modelos evaluados.

Variable	Descripción
$H_{L0.1weight}$	Volumen o peso del tráfico del host en una ventana de 0.1 s.
$H_{L0.1mean}$	Promedio del comportamiento del tráfico del host.

Variable	Descripción
$H_{L0.1variance}$	Variación o dispersión del tráfico del host.
$HH_{jitL0.01mean}$	Promedio del jitter entre host origen y host destino.
$HpHp_{L0.1radius}$	Medida de dispersión en la comunicación entre sockets.

Estas variables permiten representar aspectos como volumen, comportamiento promedio, variación, ritmo temporal y dispersión del tráfico. Dichos elementos son relevantes porque el comportamiento de red de un dispositivo IoT puede modificarse cuando es comprometido por una botnet.

Antes de presentar los resultados, es necesario distinguir entre los enfoques de aprendizaje automático utilizados en el proyecto. El aprendizaje supervisado utiliza datos etiquetados para entrenar un modelo que aprende a asociar características de entrada con una clase conocida. En este enfoque se ubica Gaussian Naive Bayes, ya que durante el entrenamiento recibe tanto las variables del tráfico como la etiqueta real de cada registro.

En cambio, el aprendizaje no supervisado trabaja sin utilizar etiquetas durante el entrenamiento. Su objetivo es encontrar estructuras, agrupamientos o patrones internos en los datos. En este enfoque se ubica K-Means, que agrupa los registros según su cercanía en el espacio de características.

K-Means se utilizó como una primera aproximación exploratoria, debido a que la tarea original del dataset también contempla clustering. El propósito de esta prueba fue observar si los registros de tráfico se agrupaban naturalmente en función de sus características estadísticas, sin proporcionar al algoritmo las etiquetas reales. Primero se probó K-Means multiclase con cinco grupos, buscando una correspondencia con las cinco clases originales. Posteriormente, se realizó una prueba binaria, reduciendo el problema a dos clases: tráfico benigno y tráfico de ataque.

Posteriormente se evaluó Gaussian Naive Bayes como modelo supervisado, con el fin de comparar el comportamiento de un clasificador que sí utiliza las etiquetas reales durante el entrenamiento. Este modelo permite estimar la probabilidad de pertenencia de una muestra a cada clase a partir de las variables observadas.

Finalmente, se agregó Isolation Forest como una prueba adicional relacionada con detección de anomalías. Esta decisión se basó en el enfoque del artículo N-BaIoT, donde los autores proponen detectar tráfico anómalo en dispositivos IoT y comparan su método con algoritmos comúnmente usados para detección de anomalías, como Local Outlier Factor, One-Class SVM e Isolation Forest.

Para medir el rendimiento de los modelos se usaron métricas de clasificación. La exactitud indica la proporción total de predicciones correctas. La precisión mide qué tan confiables son las predicciones positivas del modelo, es decir, cuántos de los registros predichos como una clase realmente pertenecen a ella. La sensibilidad o recall mide la capacidad del modelo para encontrar los casos reales de una clase; en ciberseguridad es especialmente importante porque una baja sensibilidad puede implicar ataques no detectados. El F1-score combina precisión y sensibilidad mediante una media armónica, por lo que permite evaluar el equilibrio entre ambas métricas.

Además de estas métricas, se analizaron las matrices de confusión. Una matriz de confusión permite observar aciertos y errores por clase. En el contexto de seguridad, esta tabla es fundamental porque un falso positivo puede generar una alerta innecesaria, mientras que un falso negativo representa un riesgo más grave: un ataque clasificado como tráfico benigno. Por esta razón, la evaluación no se

basó únicamente en la exactitud global, sino también en la interpretación operativa de los errores.

2.1. Métricas de rendimiento

En esta etapa se comparó el desempeño de los modelos mediante cuatro métricas principales: exactitud, precisión, sensibilidad y F1-score. Estas métricas permiten evaluar el comportamiento general de los modelos y también observar su capacidad para identificar correctamente tráfico benigno y tráfico asociado a ataques.

La exactitud representa la proporción total de predicciones correctas respecto al total de registros evaluados. La precisión indica qué tan confiables fueron las predicciones realizadas por el modelo. La sensibilidad, también conocida como recall, mide la capacidad del modelo para identificar correctamente los casos reales de cada clase. Finalmente, el F1-score resume el equilibrio entre precisión y sensibilidad, por lo que resulta útil cuando se desea evaluar el desempeño general del modelo sin depender únicamente de una sola métrica.

En el caso de K-Means, las métricas se calcularon comparando los clústeres generados contra las etiquetas reales del dataset. Debido a que K-Means no asigna nombres de clase por sí mismo, primero se realizó una correspondencia entre cada clúster y la etiqueta predominante dentro de ese grupo. Posteriormente, se calcularon las métricas de rendimiento.

Para Gaussian Naive Bayes e Isolation Forest se utilizó una división de 70% para entrenamiento y 30% para prueba. Esto permitió evaluar el rendimiento de los modelos sobre registros que no fueron utilizados durante el entrenamiento. En el caso de Gaussian Naive Bayes, el modelo fue entrenado con las etiquetas reales. En el caso de Isolation Forest, se realizó una adaptación multiclase entrenando un detector por cada etiqueta y asignando la clase con mayor puntaje de normalidad.

La siguiente tabla muestra la comparación general de los modelos evaluados:

Modelo	Exactitud	Precisión	Sensibilidad / Recall	F1-score	AUC
K-Means multiclase	0.5827	0.6201	0.5827	0.5744	N/A
K-Means binario	0.8824	0.9041	0.8824	0.8808	0.9975
Isolation Forest multiclase adaptado	0.9418	0.9602	0.9418	0.9459	0.9995
Gaussian Naive Bayes	0.9982	0.9983	0.9982	0.9983	0.9989

El AUC se reportó bajo un enfoque binario **benign** frente a **attack**. En el caso de K-Means multiclase se marca como N/A porque esta prueba fue evaluada mediante matriz de confusión multiclase y no mediante una curva ROC directa.

Los resultados muestran que K-Means multiclase tuvo el rendimiento más bajo, con una exactitud de 0.5827 y un F1-score de 0.5744. Esto indica que el algoritmo no logró separar correctamente las cinco clases originales del dataset. La causa principal es que K-Means trabaja sin etiquetas y agrupa

los registros por cercanía entre puntos, por lo que no necesariamente sus clústeres coinciden con las clases reales de ciberseguridad.

Al transformar el problema a una evaluación binaria, es decir, tráfico **benign** frente a tráfico **attack**, K-Means mejoró considerablemente. Su exactitud aumentó a 0.8824 y su F1-score a 0.8808. Esto indica que el algoritmo fue más útil para separar tráfico normal frente a tráfico anómalo en términos generales, aunque todavía presentó errores importantes que se analizan posteriormente mediante la matriz de confusión.

Isolation Forest multiclase adaptado obtuvo una exactitud de 0.9418 y un F1-score de 0.9459. Este resultado fue superior al de K-Means, lo cual sugiere que los modelos orientados a detección de anomalías pueden capturar mejor ciertos comportamientos atípicos del tráfico IoT. Sin embargo, al tratarse de una adaptación multiclase, no superó el rendimiento del modelo supervisado.

Gaussian Naive Bayes obtuvo el mejor desempeño general, con una exactitud de 0.9982, una precisión de 0.9983, una sensibilidad de 0.9982 y un F1-score de 0.9983. Este resultado se debe a que el modelo sí utiliza las etiquetas reales durante el entrenamiento, lo que le permite aprender la relación entre las variables estadísticas del tráfico y cada clase del dataset.

De forma general, las métricas muestran que el aprendizaje supervisado obtuvo el mejor rendimiento para la clasificación multiclase. No obstante, los modelos no supervisados y de detección de anomalías resultaron útiles como línea base exploratoria y como referencia para analizar escenarios donde no siempre se cuenta con etiquetas de ataque previamente conocidas.

2.2. Matrices de confusión e interpretación de errores

Además de las métricas globales, se analizaron las matrices de confusión de los modelos evaluados. En una matriz de confusión, las filas representan la etiqueta real y las columnas representan la etiqueta predicha por el modelo. Los valores ubicados en la diagonal principal corresponden a predicciones correctas, mientras que los valores fuera de la diagonal representan errores de clasificación.

Desde el punto de vista de ciberseguridad, no todos los errores tienen el mismo impacto. Un falso positivo ocurre cuando tráfico benigno es clasificado como ataque, lo cual puede generar alertas innecesarias. En cambio, un falso negativo ocurre cuando tráfico malicioso es clasificado como benigno; este error es más crítico, porque representa un ataque que podría pasar desapercibido dentro de la red.

2.2.1. Matriz de confusión de K-Means multiclase

La primera evaluación se realizó con K-Means multiclase, configurado con cinco clústeres. El objetivo fue observar si el algoritmo podía agrupar naturalmente las cinco clases originales del dataset: `benign`, `gafgyt.scan`, `gafgyt.udp`, `mirai.scan` y `mirai.udp`.

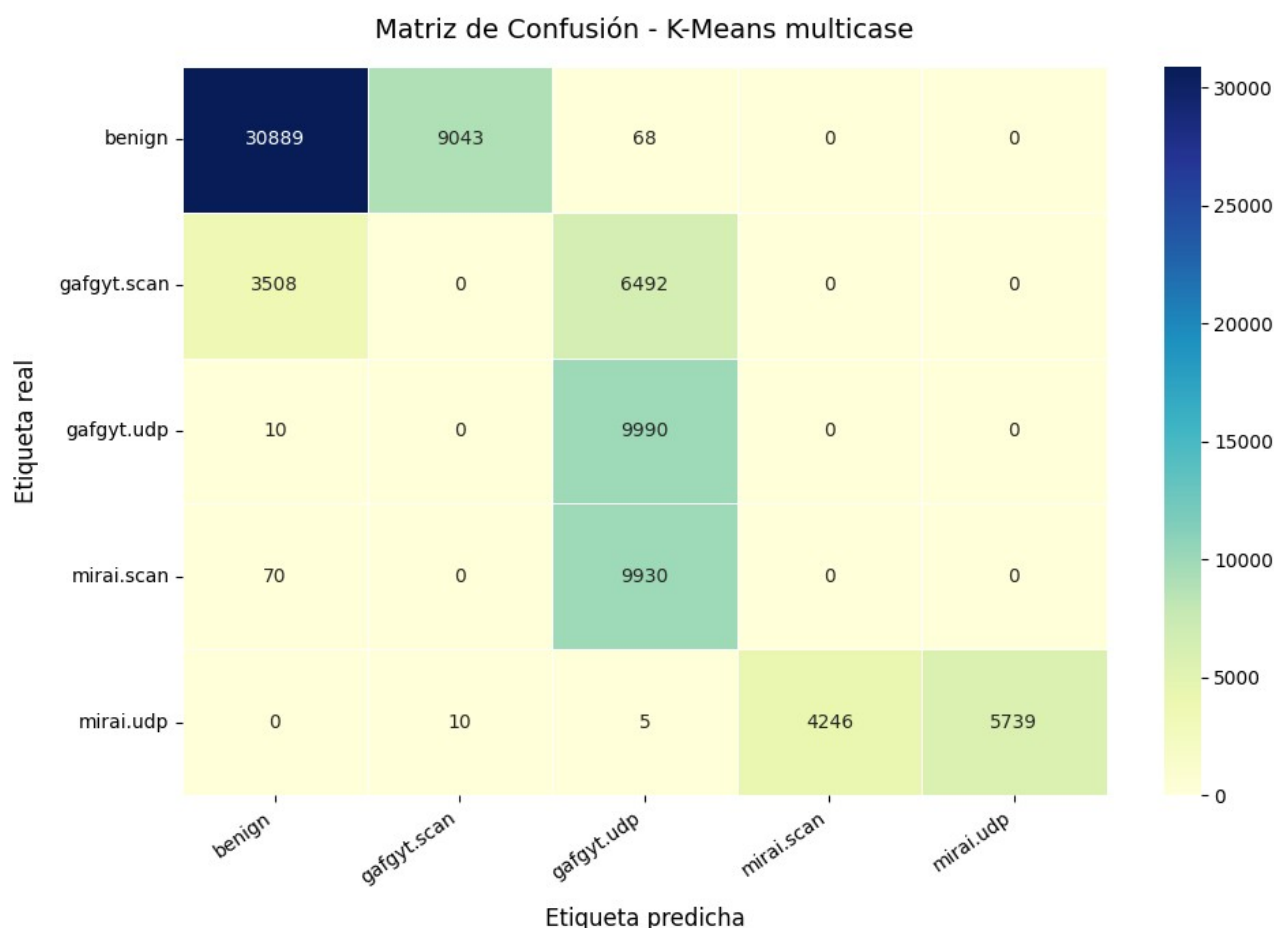


Figure 2.2.1: Matriz de confusión de K-Means multiclase.

Los resultados muestran que K-Means multiclase no logró separar correctamente las cinco clases. Aunque clasificó correctamente 30,889 registros benignos, también asignó 9,043 registros benignos como `gafgyt.scan` y 68 como `gafgyt.udp`.

El principal problema se observa en las clases de ataque. En `gafgyt.scan`, el modelo clasificó 6,492 registros como `gafgyt.udp` y 3,508 como `benign`, sin clasificar correctamente ningún registro como `gafgyt.scan`. De forma similar, la clase `mirai.scan` fue clasificada principalmente como `gafgyt.udp`, con 9,930 registros en esa categoría y solo 70 como `benign`. Esto indica que los ataques de tipo `scan` no fueron identificados correctamente por el modelo.

En el caso de `gafgyt.udp`, el modelo tuvo un mejor comportamiento, clasificando correctamente 9,990 registros como `gafgyt.udp`. Para `mirai.udp`, el modelo clasificó correctamente 5,739 registros, pero también asignó 4,246 registros como `mirai.scan`.

Estos resultados explican el bajo desempeño de K-Means multiclase. El algoritmo no utiliza etiquetas durante el entrenamiento y solo agrupa por cercanía en el espacio de características. Por ello, cuando dos tipos de tráfico presentan propiedades estadísticas similares, pueden ser agrupados dentro del mismo clúster aunque pertenezcan a clases distintas. En este caso, K-Means no fue adecuado como clasificador multiclase, pero sí permitió observar que las cinco clases no estaban separadas de forma clara bajo las variables utilizadas.

2.2.2. Matriz de confusión de K-Means binario

Después de la prueba multiclase, se aplicó K-Means en un enfoque binario, reduciendo el problema a dos categorías: **benign** y **attack**. Esta evaluación buscó analizar si el algoritmo podía separar de manera general el tráfico normal del tráfico malicioso.

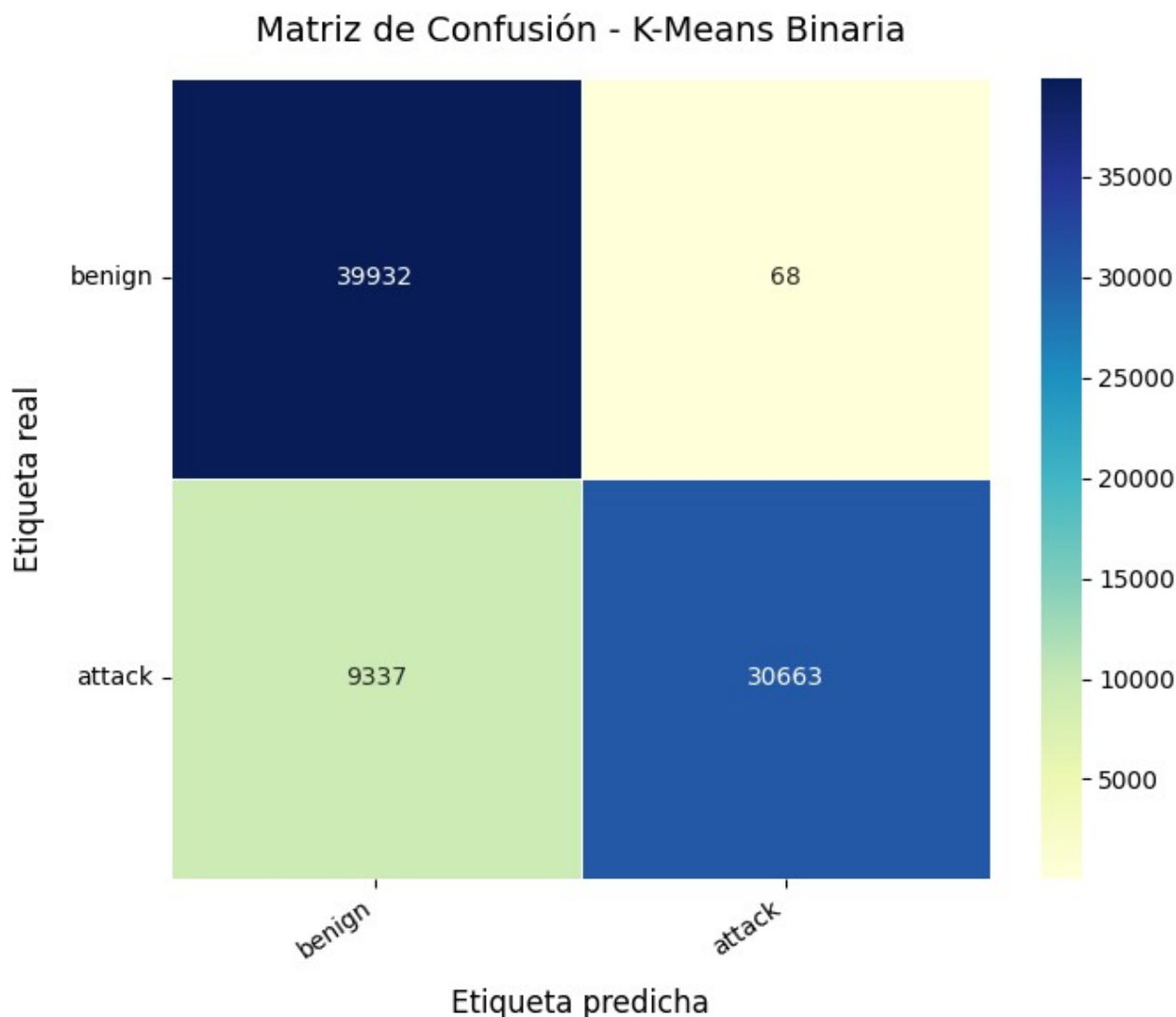


Figure 2.2.2: Matriz de confusión de K-Means binario.

En esta matriz, los 39,932 registros benignos clasificados correctamente representan el tráfico normal que el modelo identificó adecuadamente. Los 30,663 registros de ataque clasificados como **attack** representan ataques detectados correctamente.

Los 68 registros benignos clasificados como **attack** corresponden a falsos positivos. En un entorno real, estos casos podrían generar alertas innecesarias, pero no representan un compromiso directo de seguridad. En cambio, los 9,337 ataques clasificados como **benign** son falsos negativos. Este es el error más crítico, ya que indica que una cantidad considerable de tráfico malicioso habría pasado como tráfico normal.

Estos falsos negativos se explican porque K-Means no utiliza etiquetas de guía durante el entrenamiento. El algoritmo asigna cada registro al centroide más cercano mediante distancia euclidiana. Si ciertos ataques, especialmente aquellos con comportamiento más sutil, presentan

valores estadísticos parecidos al tráfico benigno de un dispositivo IoT en reposo, pueden quedar absorbidos dentro del clúster benigno. Por esta razón, aunque K-Means binario mejoró respecto a K-Means multiclase, no sería suficiente como detector principal en un entorno de ciberseguridad.

2.2.3. Matriz de confusión de Gaussian Naive Bayes

Posteriormente se evaluó Gaussian Naive Bayes como modelo supervisado multiclase. A diferencia de K-Means, este modelo sí utiliza las etiquetas reales durante el entrenamiento, por lo que aprende una relación probabilística entre las variables estadísticas del tráfico y cada clase.

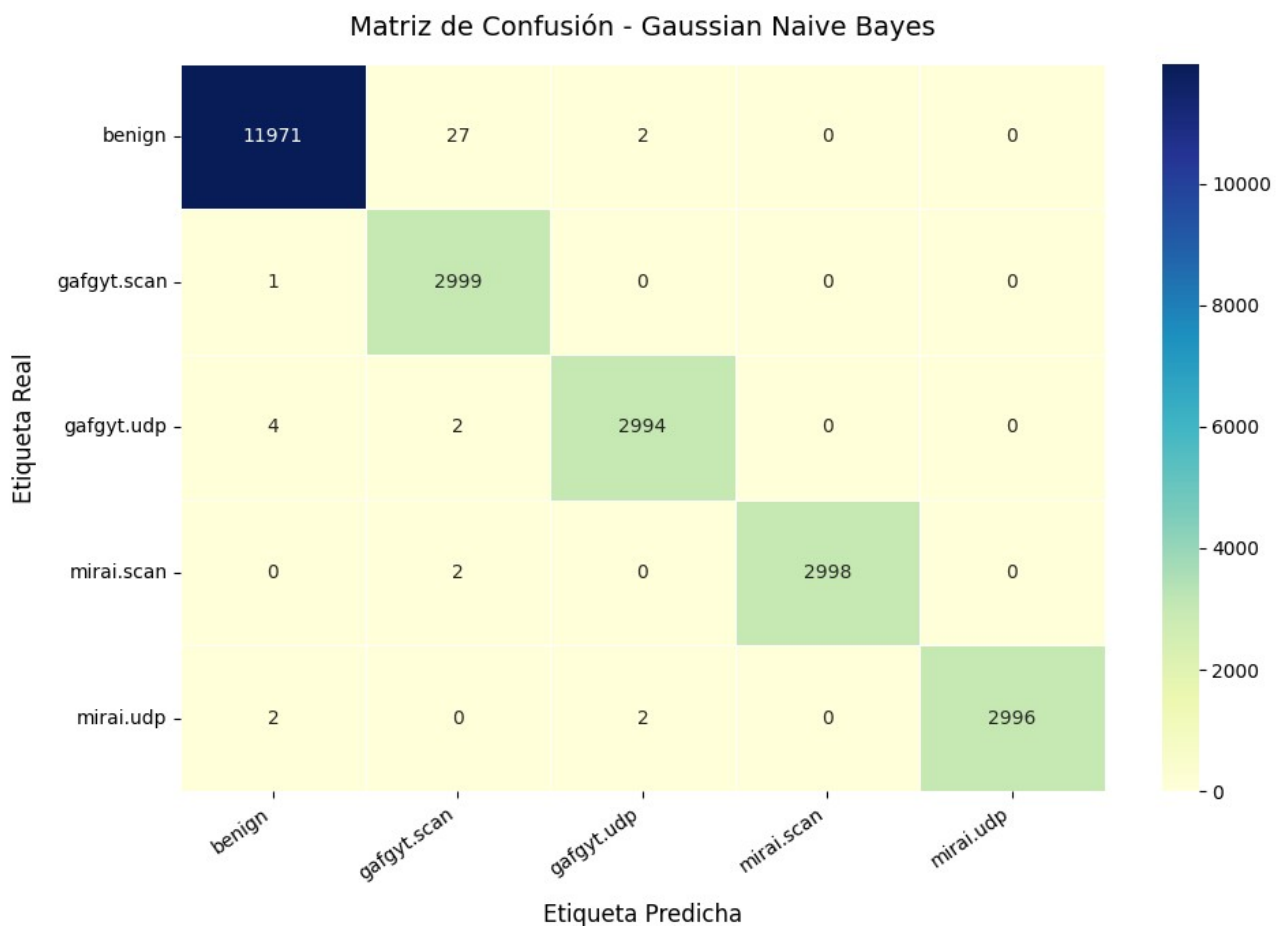


Figure 2.2.3: Matriz de confusión de Gaussian Naive Bayes.

La matriz de confusión muestra un desempeño muy alto. De los 24,000 registros del conjunto de prueba, el modelo clasificó correctamente 23,958 y cometió únicamente 42 errores.

Por clase, el modelo clasificó correctamente:

Clase real	Correctos	Total
benign	11,971	12,000
gafgyt.scan	2,999	3,000
gafgyt.udp	2,994	3,000

Clase real	Correctos	Total
mirai.scan	2,998	3,000
mirai.udp	2,996	3,000

Los errores fueron mínimos. En la clase `benign`, solo 29 registros fueron clasificados como ataques: 27 como `gafgyt.scan` y 2 como `gafgyt.udp`. En las clases de ataque, los errores también fueron muy bajos. Por ejemplo, `gafgyt.scan` tuvo solo 1 error, `gafgyt.udp` tuvo 6 errores, `mirai.scan` tuvo 2 errores y `mirai.udp` tuvo 4 errores.

Este resultado indica que Gaussian Naive Bayes fue capaz de distinguir con alta precisión entre tráfico benigno y las diferentes clases de ataque. Además, como fue evaluado sobre un conjunto de prueba separado del entrenamiento, el resultado representa el desempeño del modelo sobre datos no vistos.

2.2.4. Matriz de confusión de Isolation Forest multiclase adaptado

Finalmente, se evaluó Isolation Forest mediante una adaptación multiclase. Este algoritmo está diseñado principalmente para detección de anomalías, no para clasificación multiclase directa. Por ello, se entrenó un detector por cada etiqueta y se asignó a cada registro la clase cuyo modelo lo consideró menos anómalo.

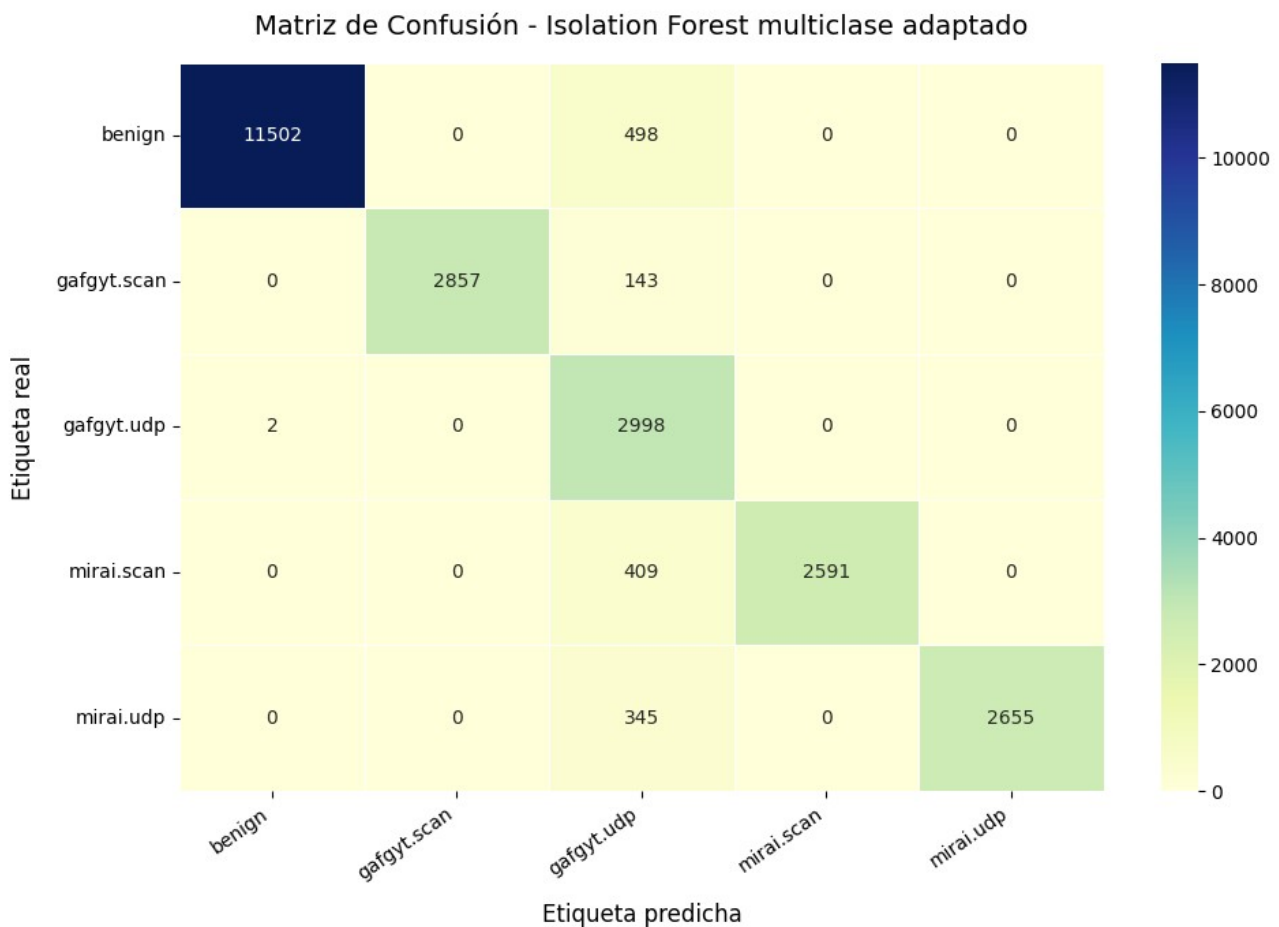


Figure 2.2.4: Matriz de confusión de Isolation Forest multiclase adaptado.

La matriz de confusión muestra que Isolation Forest obtuvo un desempeño superior al de K-Means multiclase. En la clase `benign`, clasificó correctamente 11,502 registros de 12,000, aunque 498 registros benignos fueron clasificados como `gafgyt.udp`. En `gafgyt.scan`, clasificó correctamente 2,857 registros y confundió 143 como `gafgyt.udp`. En `gafgyt.udp`, el modelo clasificó correctamente 2,998 registros de 3,000.

Sin embargo, se observaron errores relevantes en las clases `mirai.scan` y `mirai.udp`. En `mirai.scan`, el modelo clasificó correctamente 2,591 registros, pero 409 fueron clasificados como `gafgyt.udp`. En `mirai.udp`, clasificó correctamente 2,655 registros, pero 345 fueron clasificados también como `gafgyt.udp`.

Esto muestra que Isolation Forest logró una separación más clara que K-Means, pero tendió a concentrar varios errores en la clase `gafgyt.udp`. El modelo detectó muchos patrones como cercanos a esa clase, lo que explica que su precisión para `gafgyt.udp` fuera menor que en las demás clases.

Aun así, Isolation Forest alcanzó un rendimiento general alto, con una exactitud de 0.9418 y un F1-score de 0.9459. Podríamos decir que los modelos orientados a detección de anomalías pueden ser una alternativa más sólida que K-Means para analizar tráfico IoT, aunque en este caso no superaron al modelo supervisado Gaussian Naive Bayes.

2.2.5. Comparación general de las matrices

Las matrices de confusión permiten observar una evolución clara entre los modelos. K-Means multiclase tuvo dificultades para separar las cinco etiquetas originales, especialmente en los ataques de tipo `scan`. K-Means binario mejoró al reducir el problema a `benign` frente a `attack`, pero todavía dejó 9,337 ataques como benignos, lo cual representa un riesgo crítico.

Isolation Forest multiclase adaptado mejoró considerablemente frente a K-Means, pero todavía presentó confusiones hacia la clase `gafgyt.udp`. Finalmente, Gaussian Naive Bayes obtuvo la mejor matriz de confusión, con muy pocos errores en el conjunto de prueba.

El modelo más adecuado fue Gaussian Naive Bayes, debido a que redujo de forma significativa los falsos negativos y mantuvo un desempeño alto en todas las clases. Sin embargo, Isolation Forest resulta relevante como línea de mejora futura, especialmente para escenarios donde se busque detección de anomalías o donde no se disponga de etiquetas suficientes.

2.3. Curvas ROC y análisis AUC

Además de las métricas generales y las matrices de confusión, se realizó un análisis mediante curvas ROC y el área bajo la curva, conocida como AUC. La curva ROC permite observar la relación entre la tasa de verdaderos positivos y la tasa de falsos positivos cuando se modifica el umbral de decisión de un modelo.

En este proyecto, las curvas ROC se calcularon bajo un enfoque binario. Para ello, la clase `benign` se consideró como clase negativa, mientras que las clases `gafgyt.scan`, `gafgyt.udp`, `mirai.scan` y `mirai.udp` se agruparon como clase positiva `attack`. Esta conversión se realizó porque, desde una perspectiva de ciberseguridad, la primera decisión crítica consiste en determinar si el tráfico observado es benigno o malicioso.

Es importante señalar que esta evaluación ROC/AUC no sustituye la evaluación multiclase presentada en las matrices de confusión. Gaussian Naive Bayes e Isolation Forest fueron evaluados originalmente con las cinco etiquetas del dataset. Sin embargo, para comparar la capacidad general

de separación entre tráfico benigno y tráfico de ataque, se utilizó una evaluación binaria complementaria.

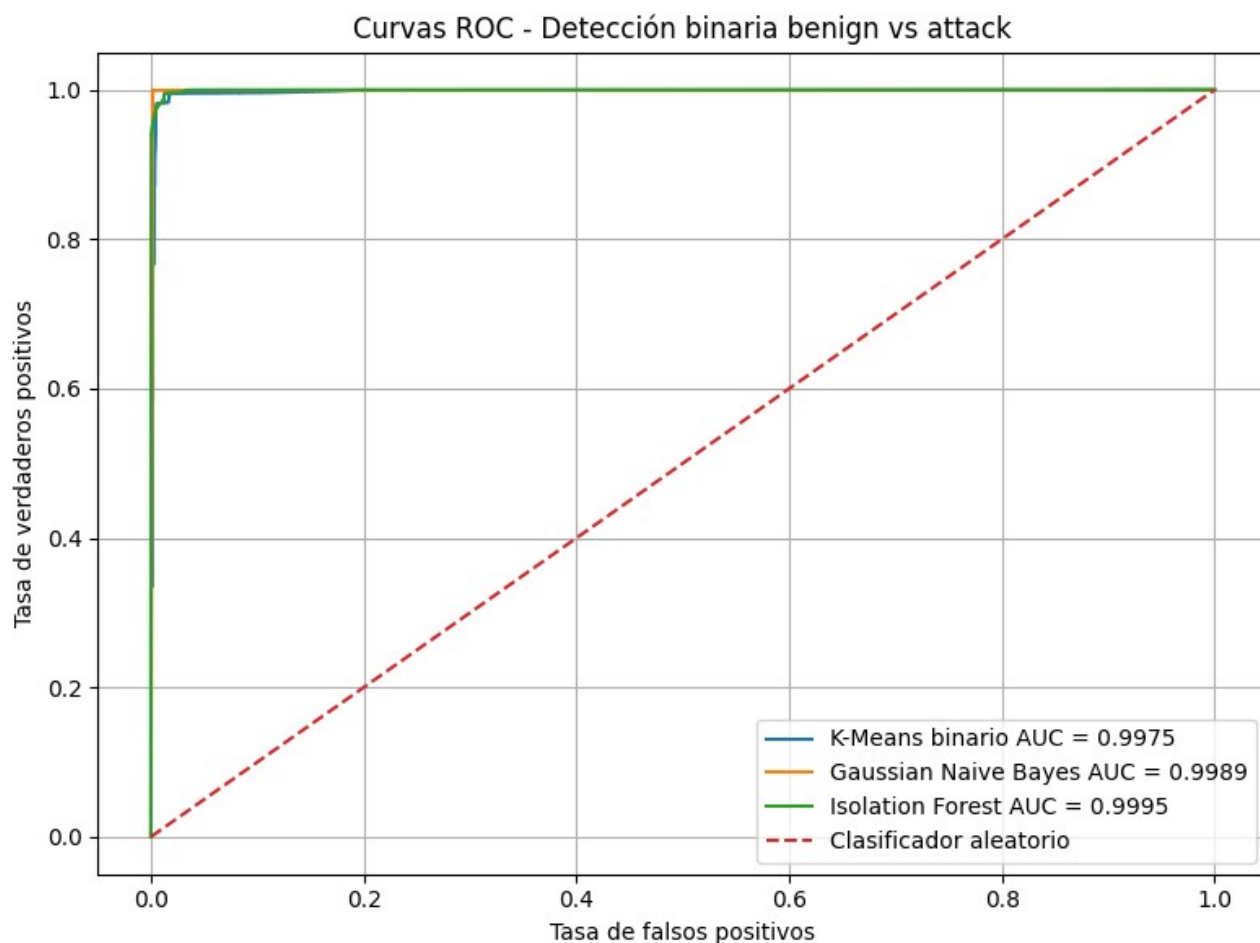


Figure 2.3.1: Curvas ROC de los modelos evaluados en detección binaria.

Los valores AUC obtenidos fueron los siguientes:

El valor AUC resume la capacidad del modelo para separar ambas clases. Un valor cercano a 1 indica una alta capacidad de discriminación entre tráfico benigno y tráfico malicioso, mientras que un valor cercano a 0.5 indica un comportamiento similar a una clasificación aleatoria.

En la gráfica se observa que los tres modelos obtuvieron valores AUC altos. Isolation Forest obtuvo el valor más alto, con 0.9995; Gaussian Naive Bayes obtuvo 0.9989; y K-Means binario obtuvo 0.9975. Esto indica que los tres modelos tienen una alta capacidad para ordenar o separar los registros benignos frente a los registros de ataque bajo una evaluación binaria.

No obstante, estos resultados deben interpretarse junto con las matrices de confusión. En el caso de K-Means binario, aunque el AUC fue alto, la matriz de confusión mostró 9,337 ataques clasificados como benignos. Esto ocurre porque la curva ROC evalúa el comportamiento del modelo ante distintos umbrales posibles, mientras que la matriz de confusión muestra el resultado final con una decisión concreta.

El AUC mide la capacidad potencial de separación del modelo, pero no reemplaza el análisis de falsos positivos y falsos negativos. En ciberseguridad, esta diferencia es importante porque un modelo puede mostrar buena separación general, pero aun así requerir ajuste de umbral para reducir ataques no detectados.

Por esta razón, el análisis ROC/AUC se utilizó como complemento de la evaluación principal. Las matrices de confusión permitieron analizar el comportamiento multiclase de los modelos, mientras que las curvas ROC permitieron comparar su capacidad general para distinguir entre tráfico benigno y tráfico malicioso.

3. Selección del modelo y justificación operativa

3.1. *Modelo seleccionado para implementación*

A partir de los resultados obtenidos, el modelo con mejor desempeño general fue **Gaussian Naive Bayes**. Este modelo alcanzó una exactitud de 0.9982, una precisión de 0.9983, una sensibilidad de 0.9982 y un F1-score de 0.9983. Además, en la matriz de confusión clasificó correctamente 23,958 de 24,000 registros del conjunto de prueba, cometiendo únicamente 42 errores.

La selección de **Gaussian Naive Bayes** como modelo más adecuado no se basa únicamente en que obtuvo el porcentaje más alto, sino en su comportamiento operativo dentro de un escenario de ciberseguridad. En este tipo de entornos, el criterio principal no es solo acertar en promedio, sino reducir la posibilidad de que tráfico malicioso sea clasificado como benigno. Un falso negativo representa un riesgo crítico, ya que permitiría que un ataque asociado a botnets pasara desapercibido dentro de la red IoT.

En comparación, **K-Means multiclase** no logró separar correctamente las cinco clases originales del dataset, debido a que es un modelo no supervisado y agrupa los registros por cercanía matemática, no por significado de seguridad. **K-Means binario** mejoró al reducir el problema a **benign** frente a **attack**, pero todavía clasificó 9,337 ataques como benignos. Este número de falsos negativos representa una limitación importante para un sistema de detección real.

Isolation Forest multiclase adaptado tuvo un rendimiento superior a **K-Means**, alcanzando una exactitud de 0.9418 y un F1-score de 0.9459. Este resultado muestra que los modelos orientados a detección de anomalías pueden ser útiles para analizar tráfico IoT. Sin embargo, al tratarse de una adaptación multiclase, todavía presentó confusiones relevantes, especialmente hacia la clase **gafgyt.udp**. Por esta razón, aunque es una alternativa interesante para trabajo futuro, no fue el modelo más sólido para una implementación inicial.

Desde una perspectiva operativa, **Gaussian Naive Bayes** resulta conveniente porque es un modelo supervisado, rápido, interpretable y de bajo costo computacional. Estas características son importantes en entornos IoT, donde los dispositivos y sistemas de monitoreo pueden tener recursos limitados. Además, al trabajar con probabilidades por clase, el modelo puede integrarse dentro de un sistema de alertamiento que indique no solo la clase predicha, sino también el nivel de confianza asociado a la predicción.

Para una implementación inicial, el modelo seleccionado sería **Gaussian Naive Bayes** como clasificador principal de tráfico. Su función sería recibir las características estadísticas extraídas del tráfico de red y clasificar cada muestra como tráfico benigno o como una de las clases de ataque analizadas. En un entorno real, este modelo podría formar parte de un sistema de monitoreo o IDS enfocado en dispositivos IoT, generando alertas cuando el tráfico sea clasificado como malicioso.

3.2. *Trade-off operativo: Modelo Escudo vs Modelo Balanceado*

En el contexto de este proyecto, la detección de botnets en dispositivos IoT exige priorizar la reducción de falsos negativos, ya que un dispositivo comprometido puede operar silenciosamente

dentro de la red, participar en ataques distribuidos o exponer información sensible del usuario. Por ello, el modelo más adecuado debe mantener una sensibilidad alta sin sacrificar de forma considerable la precisión. Bajo este criterio, **Gaussian Naive Bayes** fue el modelo más conveniente, ya que presentó un desempeño alto y equilibrado en todas las métricas evaluadas.

No obstante, la elección del modelo depende del contexto de operación. En una organización donde la prioridad absoluta sea evitar ataques no detectados, se justificaría implementar un **Modelo Escudo**. Este tipo de modelo prioriza maximizar la sensibilidad o recall, aun cuando eso incremente la cantidad de falsos positivos. Es decir, se prefiere generar más alertas antes que permitir que un ataque pase desapercibido. Este enfoque sería adecuado en hospitales, instituciones financieras, infraestructura crítica, centros de datos o redes donde una intrusión pueda tener consecuencias graves.

Por otro lado, en organizaciones donde el exceso de alertas pueda saturar al personal de seguridad, puede ser más adecuado un **Modelo Balanceado**. Este enfoque busca mantener un equilibrio entre precisión y sensibilidad, optimizando el F1-score. En este caso, el objetivo no es únicamente detectar la mayor cantidad posible de ataques, sino también reducir las falsas alarmas para que el sistema sea operativamente manejable.

Para este proyecto, el modelo seleccionado se acerca más a un enfoque balanceado con alta sensibilidad, ya que **Gaussian Naive Bayes** obtuvo valores elevados tanto en precisión como en recall. Esto significa que el modelo no solo detectó la mayoría de los ataques, sino que también redujo la cantidad de clasificaciones incorrectas. En términos operativos, esto lo vuelve más viable para una implementación inicial en un sistema de monitoreo de tráfico IoT.

Para finalizar, aunque **Gaussian Naive Bayes** fue seleccionado como el modelo principal, **Isolation Forest** representa una alternativa importante para escenarios donde no existan suficientes datos etiquetados o donde se busque detectar comportamientos desconocidos. Por esta razón, una estrategia operativa más robusta podría combinar ambos enfoques: utilizar **Gaussian Naive Bayes** como clasificador supervisado principal y complementar el monitoreo con modelos de detección de anomalías para identificar tráfico que no se ajuste a patrones conocidos.

4. Limitaciones y mejoras futuras

Aunque los resultados obtenidos muestran que los modelos de aprendizaje automático pueden apoyar la detección de tráfico anómalo en dispositivos IoT, el análisis realizado presenta limitaciones que deben considerarse antes de pensar en una implementación real. Estas limitaciones se relacionan tanto con la selección de datos como con las características propias de los algoritmos utilizados.

4.1. Limitaciones técnicas del análisis

Una primera limitación fue el número de variables utilizadas. El dataset N-BaIoT completo contiene 115 variables atributo; sin embargo, en este proyecto se trabajó únicamente con cinco características estadísticas del tráfico. Aunque estas variables permitieron obtener buenos resultados con **Gaussian Naive Bayes**, es posible que no capturen toda la complejidad del comportamiento de red de un dispositivo IoT comprometido. Algunos ataques pueden diferenciarse mejor utilizando variables adicionales relacionadas con ventanas temporales, correlación, magnitud, covarianza o comportamiento entre sockets.

Otra limitación fue el uso de un subconjunto de datos. El dataset completo contiene 7,062,606 registros, pero el análisis experimental se realizó con 80,000 registros. Esta selección permitió

trabajar de forma más controlada y eficiente dentro del entorno de Colab, pero también implica que los resultados representan únicamente una muestra del conjunto completo. En una implementación real, sería necesario validar el modelo con más registros y con diferentes dispositivos IoT.

También debe considerarse que el análisis se realizó sobre datos estáticos. Es decir, los modelos fueron entrenados y evaluados con registros previamente capturados, no con tráfico en tiempo real. En una red real, el comportamiento de los dispositivos IoT puede cambiar por actualizaciones de firmware, cambios en los hábitos de uso, variaciones de red o aparición de nuevas variantes de botnets. Esto puede provocar que un modelo entrenado con datos históricos pierda rendimiento con el tiempo.

En el caso de **K-Means**, una limitación importante es que se trata de un algoritmo basado en distancias. Esto significa que agrupa los registros según su cercanía matemática en el espacio de características, pero no comprende el significado de seguridad de cada clase. Por esta razón, algunos ataques con valores estadísticos similares al tráfico benigno pueden ser absorbidos por el clúster normal. Esta limitación se observó claramente en el modelo binario, donde 9,337 ataques fueron clasificados como tráfico benigno.

Además, **K-Means** es sensible a la escala de las variables y a la elección del número de clústeres. Aunque se aplicó normalización mediante **MinMaxScaler**, el modelo puede fallar si las clases no forman grupos compactos y separados. En el experimento multiclase, configurar `n_clusters=5` no garantizó que cada clúster correspondiera a una de las cinco etiquetas reales, ya que el algoritmo no utiliza dichas etiquetas durante el entrenamiento.

En el caso de **Isolation Forest**, la principal limitación es que su propósito original es la detección de anomalías, no la clasificación multiclase directa. Por ello, fue necesario adaptarlo entrenando un detector por cada etiqueta y asignando la clase con mayor puntaje de normalidad. Aunque esta estrategia obtuvo mejores resultados que **K-Means**, no debe interpretarse como un clasificador multiclase tradicional. Además, la matriz de confusión mostró que algunos errores se concentraron hacia la clase `gafgyt.udp`.

Por su parte, **Gaussian Naive Bayes** obtuvo el mejor desempeño, pero también tiene limitaciones teóricas. Este modelo asume independencia condicional entre las variables, es decir, supone que las características utilizadas son independientes entre sí dentro de cada clase. En tráfico de red, esta suposición puede no cumplirse completamente, ya que variables como volumen, promedio, varianza y jitter pueden estar relacionadas. Aun así, el modelo funcionó adecuadamente en este experimento, probablemente porque las clases presentan diferencias estadísticas suficientemente marcadas.

Finalmente, existe el riesgo de ruido y valores atípicos en los datos. En tráfico de red real, pueden aparecer registros anómalos que no necesariamente corresponden a ataques, sino a fallos temporales, congestión, cambios de configuración, tráfico legítimo inusual o errores de captura. Estos casos pueden afectar la precisión de los modelos y aumentar la cantidad de falsos positivos o falsos negativos.

4.2. Propuesta de mejoras y trabajo futuro

Como primera mejora, se propone ampliar el número de variables utilizadas. En lugar de trabajar solo con cinco características, se podría evaluar el conjunto completo de 115 variables del dataset N-BaIoT. También sería conveniente aplicar técnicas de selección de características para identificar cuáles aportan mayor información para distinguir entre tráfico benigno y tráfico malicioso. Esto permitiría reducir variables irrelevantes sin perder capacidad predictiva.

Otra mejora importante sería aplicar validación cruzada. En este proyecto se utilizó una división de

70% para entrenamiento y 30% para prueba. Aunque esta división permitió evaluar los modelos con datos no vistos, la validación cruzada permitiría comprobar si el desempeño se mantiene estable en diferentes particiones del dataset. Esto ayudaría a tener una evaluación más robusta y reducir la dependencia de una sola división de datos.

También se recomienda evaluar modelos adicionales orientados a detección de anomalías. En el artículo base del dataset N-BaIoT se mencionan algoritmos comúnmente utilizados para esta tarea, como **Local Outlier Factor (LOF)**, **One-Class SVM** e **Isolation Forest**. Estos modelos son relevantes porque no dependen necesariamente de una clasificación supervisada tradicional, sino que buscan identificar registros cuyo comportamiento se aleja del patrón considerado normal.

Local Outlier Factor podría utilizarse para detectar registros cuyo comportamiento local sea diferente al de sus vecinos cercanos. **One-Class SVM** permitiría aprender una frontera alrededor del tráfico considerado normal y marcar como anomalía los registros que queden fuera de esa región. **Isolation Forest**, que ya fue probado de manera adaptada en este proyecto, permite aislar observaciones anómalas mediante árboles aleatorios. Además, el paper N-BaIoT propone el uso de **autoencoders**, una arquitectura de aprendizaje profundo capaz de aprender una representación del tráfico benigno y detectar anomalías mediante el error de reconstrucción.

Estos enfoques son especialmente útiles en ciberseguridad porque pueden apoyar la detección de ataques desconocidos o variantes nuevas de botnets. En un entorno real, no siempre se cuenta con etiquetas actualizadas para cada tipo de amenaza, por lo que los modelos de detección de anomalías pueden complementar a un clasificador supervisado como **Gaussian Naive Bayes**.

Una mejora adicional consiste en ajustar los umbrales de decisión. En un escenario crítico, puede ser preferible aumentar la sensibilidad del modelo para detectar más ataques, aunque esto genere más falsos positivos. Este enfoque permitiría construir un modelo tipo **Escudo**, orientado a reducir falsos negativos. En cambio, para entornos donde el exceso de alertas sea un problema operativo, se podría configurar un modelo más balanceado, optimizando precisión y F1-score.

También sería conveniente probar estrategias de ensamble. Por ejemplo, se podría utilizar **Gaussian Naive Bayes** como clasificador principal y combinarlo con un detector de anomalías como **Isolation Forest**. De esta manera, el sistema podría clasificar ataques conocidos y, al mismo tiempo, marcar como sospechoso el tráfico que no se parezca a los patrones previamente aprendidos.

Otra línea de mejora es trabajar con datos en tiempo real. Para una implementación operativa, sería necesario construir una arquitectura capaz de capturar tráfico de red, extraer características estadísticas, normalizar los datos y enviar las muestras al modelo entrenado. Esta arquitectura podría funcionar como un sistema de detección de intrusiones orientado a dispositivos IoT.

En una etapa futura, el sistema podría incluir un proceso de aprendizaje continuo o actualización periódica del modelo. Esto permitiría adaptarse a cambios en el comportamiento del dispositivo, nuevas condiciones de red o nuevas variantes de ataque. La actualización del modelo sería importante para evitar que el sistema dependa únicamente de datos históricos.

Finalmente, como mejora de ingeniería, se propone organizar el proyecto en un repositorio con estructura modular. El repositorio debería incluir el notebook o scripts de entrenamiento, archivos de requisitos, instrucciones de ejecución, gráficas generadas, matrices de confusión y documentación del flujo experimental. Esto facilitaría la reproducción del análisis y permitiría que otros usuarios validen o extiendan el trabajo realizado.

5. Conclusiones

El proyecto permitió analizar la aplicación de modelos de aprendizaje automático para la detección de tráfico anómalo en dispositivos IoT, utilizando características estadísticas del dataset N-BaIoT. El análisis se enfocó en el tráfico del dispositivo **Provision PT-737E**, una cámara de vigilancia incluida dentro del conjunto de datos, y se trabajó con un subconjunto de 80,000 registros distribuidos entre tráfico benigno y cuatro clases de ataques asociados a botnets.

Durante el desarrollo se evaluaron diferentes enfoques. Primero se aplicó **K-Means** como modelo no supervisado, con el objetivo de observar si los datos podían agruparse naturalmente sin utilizar etiquetas reales durante el entrenamiento. La prueba multiclase mostró que K-Means no logró separar correctamente las cinco clases originales, ya que varias clases de ataque fueron mezcladas entre sí. Al reducir el problema a una clasificación binaria, **benign** frente a **attack**, el rendimiento mejoró; sin embargo, el modelo todavía clasificó 9,337 ataques como tráfico benigno. Este resultado representa un riesgo crítico en ciberseguridad, debido a que esos registros equivalen a ataques que podrían pasar desapercibidos dentro de una red IoT.

Posteriormente se evaluó **Gaussian Naive Bayes** como modelo supervisado. Este modelo obtuvo el mejor desempeño general, con una exactitud de 0.9982, precisión de 0.9983, sensibilidad de 0.9982 y F1-score de 0.9983. Además, clasificó correctamente 23,958 de 24,000 registros del conjunto de prueba. Estos resultados indican que las variables seleccionadas contienen información suficiente para distinguir de forma precisa entre tráfico benigno y las clases de ataque cuando el modelo utiliza etiquetas reales durante el entrenamiento.

También se incorporó **Isolation Forest** como prueba adicional relacionada con detección de anomalías. Aunque este modelo no está diseñado originalmente para clasificación multiclase, se adaptó entrenando un detector por cada etiqueta. Su desempeño fue superior al de K-Means, alcanzando una exactitud de 0.9418 y un F1-score de 0.9459. Esto muestra que los modelos basados en detección de anomalías pueden ser una alternativa útil para analizar tráfico IoT, especialmente en escenarios donde no se cuenta con etiquetas completas o actualizadas.

Con base en los resultados, el modelo más adecuado para una implementación inicial sería **Gaussian Naive Bayes**. Su bajo costo computacional, rapidez e interpretabilidad lo hacen viable para integrarse en un sistema de monitoreo o IDS orientado a dispositivos IoT. Su función sería recibir características estadísticas del tráfico de red y clasificar cada muestra como tráfico benigno o como una de las clases de ataque analizadas. Esta decisión no se basa únicamente en el porcentaje de exactitud, sino en su equilibrio entre precisión, sensibilidad y reducción de errores críticos.

Desde una perspectiva operativa, se propone desplegar el modelo seleccionado como parte de un sistema IDS enfocado en el segmento de red donde se encuentren los dispositivos IoT. El flujo de implementación consistiría en capturar tráfico de red, extraer características estadísticas similares a las utilizadas en el dataset, aplicar el mismo proceso de normalización y enviar las muestras al modelo **Gaussian Naive Bayes** para su clasificación. Cuando el modelo identifique tráfico malicioso, el sistema generaría una alerta para el personal de seguridad.

La implementación técnica podría realizarse mediante un servicio API ligero ejecutado en un contenedor Docker. Este servicio recibiría las características del tráfico, devolvería la clase predicha y registraría el resultado para su análisis posterior. Las alertas generadas podrían integrarse con un SIEM, un sistema de monitoreo o reglas de firewall para apoyar la respuesta ante incidentes. De esta manera, el modelo no quedaría únicamente como una prueba académica, sino como un componente que podría integrarse dentro de una arquitectura real de ciberseguridad.

No obstante, una solución real no debería depender de un único modelo. Una estrategia más robusta

sería combinar un clasificador supervisado, como Gaussian Naive Bayes, con modelos de detección de anomalías, como Isolation Forest, Local Outlier Factor, One-Class SVM o autoencoders. Esto permitiría detectar ataques conocidos mediante clasificación supervisada y, al mismo tiempo, identificar comportamientos anómalos que no coincidan con patrones previamente etiquetados.

En una arquitectura más completa, Gaussian Naive Bayes podría funcionar como clasificador principal para amenazas conocidas, mientras que un detector de anomalías marcaría comportamientos sospechosos o desconocidos. Esta combinación permitiría mejorar la postura de ciberseguridad mediante detección temprana, reducción de falsos negativos y mayor capacidad de respuesta frente a botnets conocidas y posibles variantes no etiquetadas.

El aprendizaje automático puede ser una herramienta útil para apoyar la detección temprana de botnets en dispositivos IoT. Sin embargo, su efectividad depende de la calidad de los datos, la selección de características, la correcta interpretación de los errores y la actualización continua frente a nuevas amenazas. Para trabajos futuros, se recomienda ampliar el número de variables utilizadas, evaluar el conjunto completo de registros, implementar validación cruzada, ajustar umbrales de decisión y diseñar una arquitectura de análisis en tiempo real para redes IoT.

6. Repositorio del proyecto

El código fuente, notebook, gráficas, archivos de requisitos y documentación complementaria del proyecto se encuentran disponibles en el siguiente repositorio de GitHub:

<https://github.com/jorgez2301/MachineLearning-IoT-Botnet-Detection>

7. Referencias

1. Kashif, M. (s. f.). *N-BaIoT Dataset to Detect IoT Botnet Attacks*. Kaggle. Recuperado el 20 de mayo de 2026, de <https://www.kaggle.com/datasets/mkashifn/nbaiot-dataset>
2. Meidan, Y., Bohadana, M., Mathov, Y., Mirsky, Y., Breitenbacher, D., Shabtai, A., & Elovici, Y. (2018). *N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders*. arXiv. <https://doi.org/10.48550/arXiv.1805.03409>
3. University of California, Irvine. (2018). *Detection of IoT botnet attacks N-BaIoT*. UCI Machine Learning Repository. <https://doi.org/10.24432/C5RC8J>
4. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
5. Scikit-learn developers. (s. f.). *Supervised learning*. Scikit-learn. Recuperado el 3 de junio de 2026, de https://scikit-learn.org/stable/supervised_learning.html
6. Scikit-learn developers. (s. f.). *Unsupervised learning*. Scikit-learn. Recuperado el 3 de junio de 2026, de https://scikit-learn.org/stable/unsupervised_learning.html
7. Scikit-learn developers. (s. f.). *Clustering*. Scikit-learn. Recuperado el 3 de junio de 2026, de <https://scikit-learn.org/stable/modules/clustering.html>
8. Scikit-learn developers. (s. f.). *Naive Bayes*. Scikit-learn. Recuperado el 3 de junio de 2026, de https://scikit-learn.org/stable/modules/naive_bayes.html
9. Scikit-learn developers. (s. f.). *Isolation Forest*. Scikit-learn. Recuperado el 3 de junio de 2026, de <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
10. Scikit-learn developers. (s. f.). *Metrics and scoring: Quantifying the quality of predictions*. Scikit-learn. Recuperado el 3 de junio de 2026, de https://scikit-learn.org/stable/modules/model_evaluation.html